

Food and Drink Detection Using a Fine-Tuned YOLOv26 Model

Ilijana Simonovska, Petar Trajkovski

Faculty of Electrical Engineering and Information Technologies,
University of Ss. Cyril and Methodius in Skopje, North Macedonia
simonovskailijana31@gmail.com, petartrajkovski002@gmail.com

Abstract—Ambient Intelligence (AmI) supports people unobtrusively through a continuous loop of sensing, reasoning, and acting, and health and wellbeing is among its most important application areas. Diet is a major determinant of conditions such as obesity and type 2 diabetes, yet food intake is still logged mostly by hand, which is burdensome and prone to under-reporting. Wearable smart glasses are an attractive sensing modality for dietary monitoring, as they capture egocentric images of everyday eating from the wearer’s point of view; recognizing food in this stream is the sensing-and-reasoning step that lets such a system act, for example by automatically logging intake. This paper presents a single-class food and drink detection approach based on a fine-tuned YOLOv26s model. Starting from weights pre-trained on public Kaggle food and no-food images, the model was fine-tuned on a dataset combining egocentric images from Emteq smart glasses (AFL) with conventional food and drink photographs (foodID), extended by additional Kaggle no-food and high-resolution food images. All data were annotated with the VGG Image Annotator, converted to the YOLO label format, and merged into a single training set, with every food or drink item labeled as one *food* class and everything else treated as background. The fine-tuned model was evaluated against alternative detection and segmentation architectures. The fine-tuned YOLOv26s model achieved an mAP@50 of 72%. The results show that a fine-tuned YOLO model can reliably detect food across both wearable and conventional camera images, supporting its use as a sensing component in an ambient dietary-monitoring system.

Keywords—ambient intelligence; wearable sensing; context awareness; food detection; object detection; YOLO; egocentric images; smart glasses; dietary monitoring, Intersection over Union (IoU),

I. INTRODUCTION

Ambient Intelligence (AmI) refers to digital environments that are sensitive, adaptive, and responsive to the presence of people, and that provide personalized support with little explicit effort from the user [1]. An AmI system is commonly described as a loop of three stages: it senses the environment through interconnected devices, reasons over the collected data using machine learning, and acts to adapt the environment to the user’s needs [2]. Health and wellbeing are among the most prominent AmI application domains, where continuous, unobtrusive monitoring of everyday behavior enables timely, context-aware support.

Diet is one of the everyday behaviors with the strongest effect on long-term health, and poor dietary habits are a major risk factor for obesity, type 2 diabetes, and cardiovascular

disease. Monitoring food intake is therefore valuable, but it is still done mostly by hand, through self-reported diaries or photographs that the user must label. This process is time-consuming, easily forgotten, and prone to systematic under-reporting, which limits the reliability of the resulting data. An AmI approach that observes eating automatically would remove much of this burden.

Wearable devices are a central sensing modality in AmI, and smart glasses are particularly well suited to dietary monitoring: devices such as the Emteq smart glasses capture egocentric images, recorded from the wearer’s point of view, that contain food and drinks whenever the wearer is eating. Detecting food in this image stream is the sensing-and-reasoning step that would let an ambient system act, for example by logging an eating event or providing dietary feedback. The main difficulty is that egocentric images recorded in everyday conditions vary widely in lighting, viewing angle, distance, occlusion, and motion blur, and they contain a large variety of foods and drinks, so a robust detector must separate a single but visually diverse food category from arbitrary background scenes.

Single-stage object detectors of the YOLO (You Only Look Once) family are well suited to this task, because they perform detection and segmentation in real time and can run on resource-constrained, wearable hardware. In this work we treat food and drink detection as a single-class problem: every food or drink item is labeled as the class of *food*, and everything else is treated as background, following the convention of the Ultralytics framework. We start from a YOLOv26s model that was previously trained on publicly available Kaggle food and no-food images, and we fine-tune it on a dataset that combines real egocentric images recorded with smart glasses and conventional food and drink photographs taken with a standard camera. Training on both wearable camera and mobile camera images exposes the model to a wider range of viewpoints and food appearances than glasses images alone would provide. For comparison, we additionally evaluate alternative object detection and segmentation models on the same data.

The contribution of this paper is twofold. First, we describe a complete and reproducible pipeline for food detection as an AmI sensing component, covering image annotation, conversion of the annotations to the YOLO format, merging of several data sources into a single dataset, model fine-tuning, and evaluation. Second, we report and compare the detection

performance of the fine-tuned YOLOv26 model with that of alternative detection and segmentation architectures.

II. RELATED WORK

Health and wellbeing monitoring with wearable sensors is a well-established Aml application area, covering tasks such as activity recognition, fall detection, and energy-expenditure estimation [1][2]. Dietary monitoring fits naturally within this line of work, with the additional requirement of a visual sensing modality to recognize what the wearer is eating.

Automatic recognition of food from images has been studied for over a decade and is now an established sub-field often referred to as food computing [3]. Early work focused on classifying a cropped image into one of a fixed set of dishes; the Food-101 dataset [4] and the UEC-Food collections [5] became standard benchmarks for this task. Localizing food within an uncropped image is harder, and Bolaños and Radeva [6] show that food regions and their categories can be detected simultaneously with a convolutional network. More recent systems build on single-stage detectors of the YOLO family [7], implemented in widely used frameworks such as Ultralytics [8], which predict bounding boxes or segmentation masks directly and run fast enough for real-time and wearable use. The present work follows this direction: we apply a fine-tuned YOLO detector to egocentric and conventional images and treat food as a single class, so that the detector can serve as the sensing component of an ambient dietary-monitoring system.

III. DATA

The dataset used in this study was assembled from two separate sources and extended with additional images collected from Kaggle. All images were annotated with a single class, *food*, which covers both food and drink items; any region that does not contain food is treated as background.

The first source is the AFL dataset, which consists of real egocentric images recorded with Emteq smart glasses. Because these images were captured by a wearable device during everyday activity, they reflect the realistic conditions in which the final Aml system is expected to operate, including natural lighting, varied camera angles, and partial occlusion of the food.

The second source is the foodID dataset, which contains conventional food and drink photographs taken with a standard (mobile) camera. To increase the diversity of the training data and to reduce false detections on non-food scenes, the dataset was extended with two additional sets of Kaggle images: a set of no-food images, which provide negative (background-only) examples, and a set of high-resolution food images, which add visual detail and a wider range of food appearances.

All images were annotated with the VGG Image Annotator (VIA) [9], a lightweight, browser-based tool for drawing regions around objects of interest. The regions were exported from VIA as CSV and converted to the YOLO label format, in which each bounding box is stored on a single line of a

plain-text file as a class index followed by normalized coordinates. Each source dataset was split into training, validation, and test subsets, which were then merged into a single final dataset so that the model is trained and evaluated on images from all sources together. The final dataset contains 2,339 images, split into 1,870 training, 233 validation, and 236 test images (an approximate 80/10/10 split).

IV. METHODS

Computer vision is a field of artificial intelligence whose goal is the automatic extraction of information from images and video recordings. One of its most applicable capabilities is object detection. Unlike image classification models, in which a class label is assigned to the entire image, in the process of detection both the class affiliation and the position of each relevant object or region are determined simultaneously. The position of the object is most often represented by a rectangular frame, known as a bounding box. When preparing the data, the bounding box can be defined by the coordinates of its center and its dimensions:

- X- coordinate value,
- Y- coordinate value,
- W- width of the bounding box;
- H- height of the bounding box.

Note: These values are in the range between 0 and 1, they are normalized in terms of the image dimensions.

The models are evaluated by comparing the bounding box predicted by the model with the original annotated (ground truth) bounding box. For this purpose, the Intersection over Union (IoU) metric is used, which measures the similarity between the two bounding boxes. IoU calculates the ratio between the area of their intersection and the area of their union, with values ranging from 0 to 1. The higher the IoU value, the greater the overlap between the predicted and reference bounding boxes, indicating more accurate object localization.

In this paper, we utilized the YOLOv26 model architecture [10] for object detection and localization. The name YOLO is an abbreviation for You Only Look Once. The main idea is that the entire image is processed in a single pass through the series of convolutional neural networks. Instead of first extracting potential regions and then analyzing each region separately, YOLO directly predicts the bounding boxes and the corresponding class probabilities from the input image. This reduces inference time.

The architecture of YOLOv26 for object detection can be divided into three main components:

- Backbone: used for extracting visual features from the image;
- Neck: detecting more detailed features in the images and combining them with features extracted from earlier layers
- Detection head: predicting classes and bounding boxes;

The standard YOLOv26 configuration detects features at three spatial levels: P3/8, P4/16 and P5/32.

The /x notation indicates that the spatial dimensions of the feature maps are reduced by a factor of x, namely 8,16,32 respectively. relative to the input image. The P3 map has the highest spatial resolution and is especially important for detecting smaller objects. The P5 map has a lower spatial resolution, but this layer detects features related to larger objects in the original image .

- YOLOv26 is available in a number of variants:
- Nano (YOLOv26n): the smallest and fastest model.
 - Small (YOLOv26s): a smaller model with solid accuracy;
 - Medium (YOLOv26m): a balance between accuracy and complexity;
 - Large (YOLOv26l): a larger model with higher accuracy
 - Extra Large (YOLOv26x): the largest model with the highest demand for computational resources.

The prefix in the model name defines the variant of the model itself. In the two largest models, the C3k2 block is executed twice, while in the other models, it appears only in one instance. In the convolutional layers, the number of kernels varies depending on the model variant and increases proportionally with the size of the model. A larger number of kernels enables the detection of a greater number of features in the images. Figure 1.a presents the complete YOLOv26 architecture, while Figure 1.b shows a table of the YOLOv26 variants and their corresponding properties.

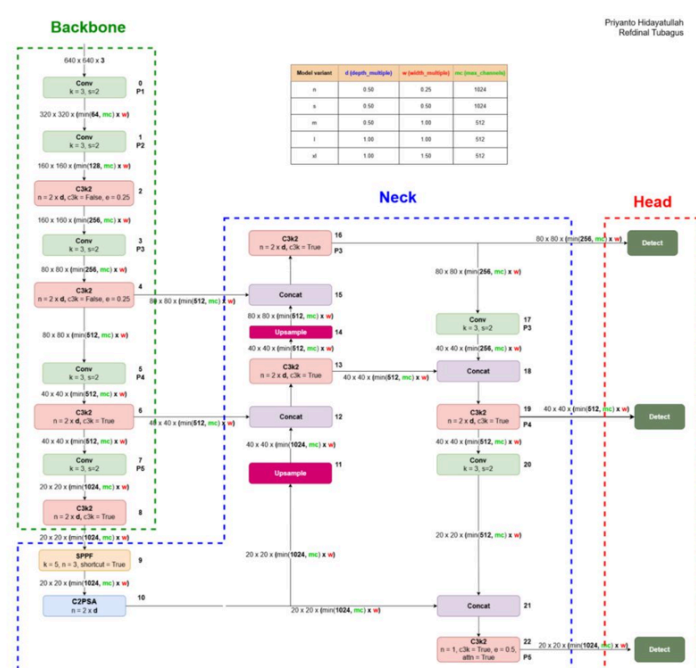


Figure 1.a: Overview of the YOLOv26 architecture

Model variant	d (depth_multiple)	w (width_multiple)	mc (max_channels)
n	0.50	0.25	1024
s	0.50	0.50	1024
m	0.50	1.00	512
l	1.00	1.00	512
xl	1.00	1.50	512

Figure 1.b: Variants of the YOLOv26 architecture and its properties

The backbone part is responsible for gradually transforming the input image into a set of feature maps. In the initial layers, simple visual elements are extracted, such as edges, lines and changes in illumination. In the deeper layers, more complex structures are recognized, such as plates, drinks, glasses and actual food.

This part is composed of convolutional layers with a kernel size of 3x3 and stride equal to 2. The output of each of these layers has dimensions that are twice as small as those of the input, while the number of output feature maps is equal to the number of kernels in the convolutional layer. Additionally, several C3k2 blocks are placed in the backbone part. The purpose of these blocks is to extract additional image features while preserving part of the previous feature with an efficient number of computations.

The Spatial Pyramid Pooling Fast Layer (SPPF) layer is used to combine information from different effective receptive fields. Through the consecutive application of max-pooling operations, the model can analyze features in different spatial contexts. In this way, the ability to recognize structures of different sizes is improved, without significantly increasing computational complexity.

Deeper in the backbone part, the C2PSA module is also included. The module applies an attention mechanism, which allows the model to assign greater importance to visually relevant parts of the feature map. Instead of all positions in the image contributing equally, the attention mechanism helps the model focus more effectively on the parts that contain useful information.

After the extraction of features, the information is forwarded to the neck part of the model . Its task is to combine the features extracted from deeper layers, which contain more semantic information, with the features from earlier layers which preserve geometric characteristics. The Upsample block increases the resolution of the feature maps. The obtained map is then merged with the corresponding map from the backbone part using the Concat operation. After this process is completed, the maps are sent for additional processing. The higher-resolution maps are passed through a convolutional layer with stride equal to 2. The features are then merged again using the Concat operation. As a result three enriched feature maps are obtained:

- P3/8 - features for smaller regions;
- P4/16 - features for medium-sized regions;
- P5/32 - features for larger regions.

The final part of the model is the Detect module. It receives the features from the three levels and converts them into final predictions. For each detection, the model determines the coordinates of the bounding box and the probability of the predicted class. Unlike other models, YOLOv26 allows Non-Maximum Suppression to be disabled, meaning the step for removing overlapping boxes over the same object can be omitted, therefore the inference speed can be increased.

Additionally, the Distribution Focal Loss (DFL) module from the head used for the bounding box estimation is removed from the detection head. In previous approaches, the distances to the sides of the bounding box are represented through discrete distributions of possible values, after which the final coordinates are calculated based on these distributions. YOLOv26 applies a simpler direct regression approach, which reduces the complexity of the detection head and reduces inference overhead.

V. RESULTS AND DISCUSSION

To evaluate the model, we used accuracy, precision, recall and F1-score. We also trained and tested the model with three different input shapes, such as 640x640, 768x768 and 960x960. Our goal was to identify the input size that provides the best balance between precision, recall, and F1-score.

Input shape	TP food	FP food	FN food	Precision	Recall	F1-score
640x640	190	46	59	80.5%	76.3%	78.4%
768x768	197	41	52	82.8%	79.1%	80.9%
960x960	199	45	50	81.6%	79.9%	80.7%

Table 1: Evaluation metrics of the three variants

After calculating the confusion matrices of all variants (Table 1), we saw that the first variant of the model correctly detects 190 images containing food, but it also produces 46 false positive predictions and 59 cases in which food is not detected. In the second matrix, we noticed a slight increase in correct predictions. The model correctly detected 197 food images, while the number of false positive predictions decreased to 41 and the number of missed food images was 52. Furthermore, the $mAP@50$ value of 0.72 shows moderate detection performance at $IoU=0.50$, while the

lower $mAP@50-95$ value of 0.49 indicates that localization becomes weaker under stricter overlap requirements. The third matrix has the highest number of correctly detected food images, with 199 true detections, and the lowest number of missed detections, with 50 cases. However, it also produced a higher number of false positives, namely 45. The $mAP@50$ and $mAP@50-95$ values of the first and third variant were both 0.71 and 0.506 respectively.

Therefore, the second configuration can be considered the best overall, because it achieves the best balance between precision and recall. It has the lowest number of incorrect detections on background images and the highest F1-score, which indicates that the model is more stable and reliable when distinguishing between images with food and images without food. The third configuration is better if the main goal is to miss as few food images as possible, but due to the number of false positive predictions, the second configuration represents a better choice for general use.

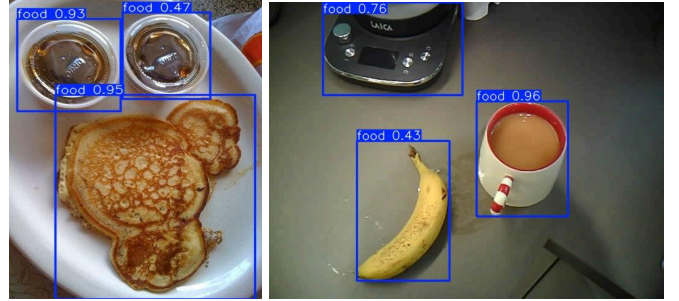


Figure 2.a and 2.b: A couple of samples from the predictions subset

VI. CONCLUSION

In this paper, we fine-tuned a YOLOv26s model for image analysis, object detection and localization. After evaluating the final model, we concluded that it reaches its best performance when the input tensor shape is transformed into a new image with width and height equal to 768 pixels due to the variety of image sizes in the entire dataset. The results show that the model performs reasonably well in detecting food objects in images. In examples like Figure 2.b, the model sometimes struggles to ignore non-edible objects. One potential reason is the absence of images that do not contain food but contain kitchen utensils, power cords, silverware and similar objects. Additionally, we attempted to add a small amount of data augmentation during the training process, but the results did not improve significantly.

A potential solution is to extend the original dataset by adding pictures that contain only the objects that the model incorrectly detected as food. A simple adjustment, such as lowering the confidence threshold can help in reducing the number of false negatives, but there is a risk of a false positives increase. Due to the limitations of our hardware setup, we could not experiment with extensive data

augmentation as it requires a lot of video memory to process the image changes.

REFERENCES

- [1] D. J. Cook, J. C. Augusto, and V. R. Jakkula, "Ambient intelligence: Technologies, applications, and opportunities," *Pervasive and Mobile Computing*, vol. 5, no. 4, pp. 277–298, 2009. <https://doi.org/10.1016/j.pmcj.2009.04.001>
- [2] Z. Mahmood (Ed.), *Guide to Ambient Intelligence in the IoT Environment: Principles, Technologies and Applications*. Springer, 2019. <https://doi.org/10.1007/978-3-030-04173-1>
- [3] W. Min, S. Jiang, L. Liu, Y. Rui, and R. Jain, "A Survey on Food Computing," *ACM Computing Surveys*, vol. 52, no. 5, 2019. <https://arxiv.org/abs/1808.07202>
- [4] L. Bossard, M. Guillaumin, and L. Van Gool, "Food-101 – Mining Discriminative Components with Random Forests," in *ECCV*, 2014. https://data.vision.ee.ethz.ch/cvl/datasets_extra/food-101/
- [5] Y. Kawano and K. Yanai, "Automatic Expansion of a Food Image Dataset Leveraging Existing Categories with Domain Adaptation," *ECCV Workshops*, 2014 (UEC-Food256). <http://foodcam.mobi/dataset256.html>
- [6] M. Bolaños and P. Radeva, "Simultaneous Food Localization and Recognition," in *ICPR*, 2016. <https://arxiv.org/abs/1604.07953>
- [7] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *CVPR*, 2016. <https://arxiv.org/abs/1506.02640>
- [8] Ultralytics, "YOLO Documentation" (training, fine-tuning, segmentation, and the YOLO label format). <https://docs.ultralytics.com>
- [9] A. Dutta and A. Zisserman, "The VIA Annotation Software for Images, Audio and Video," in *ACM Multimedia*, 2019. <https://arxiv.org/abs/1904>
- [10] P. Hidayatullah and R. Tubagus, "YOLOv26: A Comprehensive Architecture Overview and Key Improvements," *arXiv preprint arXiv:2602.14582*, 2026. doi: 10.48550/arXiv.2602.14582. <https://arxiv.org/abs/2602.14582>